

Gello 双臂

电脑环境：ubuntu20.04

ros版本：ros noetic

代码仓库：

采集部署：<https://github.com/gen-robot/realRL.git>

训练：<https://github.com/emia7/openpi.git>

双臂环境setup（目前沿用qiuyi姐setting）

ROS多机

- 主4090
 - Hostname: ubuntu-franka-master
 - ip: 192.168.120.43
- 从4090
 - Hostname: ubuntu-franka-slave
 - ip: 192.168.120.42

添加对方IP和主机名

代码块

```
1 sudo vim /etc/hosts
```

ubuntu-franka-master中

```
1 192.168.120.42 ubuntu-franka-slave
2
3 127.0.0.1 localhost
4 127.0.1.1 ubuntu-franka-master
```

ubuntu-franka-slave中

```
1 192.168.120.43 ubuntu-franka-master
2
```

```
3 127.0.0.1 localhost
4 127.0.1.1 ubuntu-franka-slave
```

环境变量增加ROS配置

ubuntu-franka-master中~/.bashrc中增加

```
1 export ROS_HOSTNAME=ubuntu-franka-master
2 export ROS_MASTER_URI=http://ubuntu-franka-master:11311
```

ubuntu-franka-slave中~/.bashrc中增加

```
1 export ROS_HOSTNAME=ubuntu-franka-slave
2 export ROS_MASTER_URI=http://ubuntu-franka-master:11311
```

Gello安装

一、安装 Polymetis

1. 克隆仓库

代码块

```
1 cd ~
2 git clone <faio 仓库地址>
```

2. 创建环境

代码块

```
1 cd ~/faio/polymetis
2 conda env create -f ./polymetis/environment.yml
3 conda activate polymetis-local
```

3. 安装 Python 包

代码块

```
1 pip install -e ./polymetis
```

4. 从源码安装 libfranka

代码块

```
1 cd ~/fairo/polymetis
2 ./scripts/build_libfranka.sh 0.19.0
```

若出现 `ParentJoint` 报错，需要修改以下文件

文件路径：

代码块

```
1 ~/fairo/polymetis/polymetis/src/clients/franka_panda_client/third_party/libfranka/src/robot_model.cpp
```

修改内容如下。

第 32 行：将 `.parentJoint` 改为 `.parent`

代码块

```
1 last_joint_index_ = // NOLINT(cppcoreguidelines-prefer-member-initializer)
2 pinocchio_model_.frames[last_link_frame_index_]
3 .parent;
```

第 55 行：将 `parent_frame.parentJoint` 改为 `parent_frame.parent`

代码块

```
1 pinocchio::FrameIndex new_frame_index =
  pinocchio_model_.addFrame(pinocchio::Frame(
2     name, parent_frame.parent, parent_frame_id, placement,
  pinocchio::OP_FRAME));
```

5. 从源码安装 Polymetis

代码块

```
1 cd ~/fairo/polymetis
2 mkdir -p ./polymetis/build
3 cd ./polymetis/build
4
```

```
5  cmake .. -DCMAKE_BUILD_TYPE=Release -DBUILD_FRANKA=ON -DBUILD_TESTS=OFF -
    DBUILD_DOCS=OFF
6  make -j
```

若编译失败

很可能是因为 conda 中安装了一个 `pinocchio`，而 ROS 中也有一个，导致 CMake 链接混乱。可按如下方式重新编译。

代码块

```
1  # 1. 清理旧的构建文件（必须做）
2  cd ~/fairo/polymetis/polymetis
3  rm -rf build/
4  mkdir build
5  cd build
6
7  # $CONDA_PREFIX 会自动指向 /home/ubuntu/miniconda3/envs/polymetis-local
8  cmake .. \
9      -DCMAKE_BUILD_TYPE=Release \
10     -DCMAKE_PREFIX_PATH=$CONDA_PREFIX \
11     -DCMAKE_NO_SYSTEM_FROM_IMPORTED=ON \
12     -DBUILD_FRANKA=ON \
13     -DBUILD_TESTS=OFF \
14     -DBUILD_DOCS=OFF
15
16  make -j
```

6. 测试安装是否成功

代码块

```
1  conda activate polymetis-local
2  launch_robot.py robot_client=franka_hardware
```

二、安装 gello-software

1. 克隆仓库

代码块

```
1  cd ~
```

```
2 git clone <gello_software 仓库地址>
```

2. 直接使用 `polymetis-local` 环境

代码块

```
1 conda activate polymetis-local
```

3. 安装 Python 包

代码块

```
1 pip install -r requirements.txt
2 pip install -e .
3 pip install -e third_party/DynamixelSDK/python
```

三、IP 配置与端口配置

一共需要启动 **4 个终端**。

说明：终端 1 和 2 不需要进入对应路径。

终端 1：启动 Polymetis 的 robot server

代码块

```
1 conda activate polymetis-local
2 launch_robot.py robot_client=franka_hardware
```

关闭后建议彻底杀死进程，否则下次启动可能报错：

代码块

```
1 sudo pkill -9 run_server
```

- 通信端口： `50051`

终端 2：启动 Polymetis 的 gripper server

```
代码块
1 conda activate polymetis-local
2 launch_gripper.py gripper=robotiq_2f
```

- 通信端口： 50052

说明

上述两个服务由 Polymetis 自己的 C++ 代码通信，之后再通过回调的方式调用 `libfranka`。

注意：在终端 3、4 启动后，gello 就会与 Franka 建立连接，需要注意安全。

终端 3：启动接收 gello 信号的 `launch_nodes.py`（gello 的 Server）

代码块

```
1 conda activate polymetis-local
2 cd ~/gello_software/experiments
3 python launch_nodes.py --robot=fr3 --robot_ip=127.0.0.1
```

此时机器人会运动到设定的初始位姿。

如果启动后遥杆操作终端尚未启动，终端会持续报 `timeout` 错误，这种情况可以暂时忽略。

- 通信端口： 5555
- 用途：与下面的 `run_env.py` 通信

对应默认参数如下：

代码块

```
1 @dataclass
2 class Args:
3     robot: str = "xarm"
4     robot_port: int = 5555
5     hostname: str = "127.0.0.1"
6     robot_ip: str = "172.16.0.2"
```

终端 4：启动发送 gello 信号的 `run_env.py`（gello 的 Client）

启动前请先握紧主臂夹爪，并将主臂抬至与 Franka 随动臂相近的姿态。

注意：此时机械臂可能会快速运动到遥杆所处位姿，请务必注意人员安全。

代码块

```
1 conda activate polymetis-local
2 cd ~/gello_software/experiments
3 python run_env.py --agent=gello
```

- 通信端口： 5555
- 用途：与上面的 `launch_nodes.py` 通信

对应默认参数如下：

代码块

```
1 @dataclass
2 class Args:
3     agent: str = "none"
4     robot_port: int = 5555
5     wrist_camera_port: int = 5000
6     base_camera_port: int = 5001
7     hostname: str = "127.0.0.1"
```

四、注意事项

0. 尽量不要使用代理

代理可能会影响 gRPC 通信。

1. 启动 `robot_client` 前，需要正确设置机械臂控制 IP

该 IP 即浏览器访问的 IP，例如：

文件路径：

代码块

```
1 ~/faipo/polymetis/polymetis/conf/robot_client/franka_hardware.yaml
```

修改参数：

代码块

```
1 robot_ip: "172.16.0.2"
```

2. 启动 gripper 前，也需要正确设置机械臂控制 IP

文件路径：

代码块

```
1 ~/faipo/polymetis/polymetis/conf/gripper/franka_hand.yaml
```

修改参数：

代码块

```
1 robot_ip: "172.16.0.2"
```

3. 若在相同位姿下一直无法启动 `run_env.py`

可检查并修改设备序列号及关节偏移配置。

配置文件：

代码块

```
1 ~/gello_software/gello/agents/gello_agent.py
```

参考配置如下：

其中 `/dev/serial/by-id/usb-FTDI_USB__-__Serial_Converter_FTAKC4NZ-if00-port0` 需要到 `/dev/serial/by-id` 目录下查看并替换为当前设备实际端口。

代码块

```
1  "/dev/serial/by-id/usb-FTDI_USB__-__Serial_Converter_FTAKC4NZ-if00-port0":  
    DynamixelRobotConfig(  
2      joint_ids=(1, 2, 3, 4, 5, 6, 7),  
3      joint_offsets=(  
4          0 * np.pi / 2,  
5          2 * np.pi / 2,  
6          2 * np.pi / 2,  
7          2 * np.pi / 2,  
8          2 * np.pi / 2,  
9          2 * np.pi / 2,  
10         5 * np.pi / 4,  
11     ),  
12     joint_signs=(1, -1, 1, -1, 1, 1, 1),  
13     gripper_config=(8, 116, 74),
```


4. 第一次使用时需要做详细标定

整体思路与上一步类似。

代码块

```
1  conda activate polymetis-local
2  cd ~/gello_software/scripts
3  python3 gello_get_offset.py \
4      --start-joints 0 -1.57 1.57 -1.57 -1.57 0 \
5      --joint-signs 1 1 -1 1 1 1 \
6      --port /dev/serial/by-id/usb-FTDI_USB__-__Serial_Converter_FT7WBG6
```

4.2 参数说明

- `--start-joints`：启动操作时机器人所处的初始位姿，可自行设置，单位为弧度，参数个数与机械臂自由度一致。
- `--joint-signs`：机器人运动方向符号。Franka 可暂时采用：

代码块

```
1  1 -1 1 1 1 -1 1
```

- `--port`：遥杆端口号，可通过 `/dev/serial/by-id/` 查看当前设备端口。

4.3 应用标定结果

运行上面的命令后，会输出 `gello-offset` 值。

需要将该值写入：

代码块

```
1  ~/gello_software/gello/agents/gello_agent.py
```

4.4 验证标定参数

按照完整遥操流程启动各个进程后，在启动 `run_env.py` 时会显示：

- 遥杆位姿
- 机器人当前姿态

- 两者之间的误差

根据该误差，继续修改 `gello_agent.py` 中对应关节的 `start-joints` 值，直到：

1. 可以正常启动遥控；
2. 摇杆操作时机器人各关节能够小幅跟随；
3. 摇杆关节运动方向与机器人关节方向一致。

若方向不一致，则修改对应的 `joint-signs`。

说明：

- 关节序号：`joint[0-6]`
- 夹爪开闭宽度：`joint[7]`

5. 机器人初始位姿配置

启动机器人后，机器人的初始位姿可通过配置文件修改。

配置文件：

代码块

```
1 /fairo/polymetis/polymetis/conf/robot_model/franka_panda.yaml
2 /fairo/polymetis/polymetis/conf/robot_model/franka_panda_with_hand.yaml
```

修改其中的：

代码块

```
1 rest_pose
```

为期望的初始位姿即可。

6. 各进程 IP 与端口配置

6.1 Franka IP 配置

修改文件：

代码块

```
1 ~/fairo/polymetis/polymetis/conf/robot_client/franka_hardware.yaml
```

修改参数：

代码块

```
1 robot_ip
```

该值应设置为 Franka FCI 的固定 IP。

6.2 Franka Hand IP 配置

修改文件：

代码块

```
1 ~/faio/polymetis/polymetis/conf/gripper/franka_hand.yaml
```

修改参数：

代码块

```
1 robot_ip
```

该值应设置为 Franka FCI 的固定 IP。

6.3 Polymetis 端口配置

修改以下文件中的 `port` 值：

代码块

```
1 ~/faio/polymetis/polymetis/conf/launch_robot.yaml
2 ~/faio/polymetis/polymetis/conf/launch_gripper.yaml
```

6.4 `launch_nodes.py` 中的 IP 与端口配置

文件路径：

代码块

```
1 ~/gello_software/experiments/launch_nodes.py
```

需要注意：

- `hostname`：主机 IP

- `robot_ip`：运行 `robot_client` 进程的上位机 IP
- `port`：不能与 `robot_client` 和 `gripper` 的端口相同

6.5 `run_env.py` 中的 IP 与端口配置

文件路径：

代码块

```
1 ~/gello_software/experiments/run_env.py
```

需要注意：

- `hostname`：主机 IP
- `port`：需与 `launch_nodes.py` 中保持一致

6.6 抓手说明

配置文件中包含 `robotiq_2f_85` 的相关配置。

如果后续更换抓手，可以参考对应配置方式进行修改。

7. 整体调试流程建议

建议遵循以下顺序：

1. 先将 gello 的初始位置摆到与 Franka 初始位置一致；
2. 运行校正脚本，得到 offset；
3. 将校正结果更新到对应配置文件；
4. 按照文档中的遥操流程完整运行；
5. 若发现有关节运动方向相反，修改 `sign`；
6. 修改后重新进行校正。

采集过程

硬件拓扑

左臂主机：

- Franka（左臂）
- 左腕 **Lumos**

- 左腕 **D435i (RealSense)**
- 第三视角 **D435i (RealSense)**
- 运行：gello 控制（左臂）、左臂状态发布、左侧 3 路相机发布、realRL recorder 采集落盘

右臂主机：

- Franka（右臂）
- 右腕 **Lumos**
- 右腕 **D435i (RealSense)**
- 运行：gello 控制（右臂）、右臂状态发布、右侧 2 路相机发布

端口规划

右臂状态（右臂主机发布）

- `right_arm_state` : `tcp://0.0.0.0:6001`

左臂状态（左臂主机发布）

- `left_arm_state` : `tcp://0.0.0.0:6000`

相机（按 topic 分端口发布）

- 左臂主机：
 - `third_d455` : `tcp://0.0.0.0:7000`
 - `left_wrist_lumos` : `tcp://0.0.0.0:7001`
 - `left_wrist_d435` : `tcp://0.0.0.0:7002`
- 右臂主机：
 - `right_wrist_lumos` : `tcp://0.0.0.0:7003`
 - `right_wrist_d435` : `tcp://0.0.0.0:7004`

左臂（主臂）

1. 起gello控制

代码块

```
1 # 终端1:起Polymetis robot server (端口 50051)
2 conda activate polymetis-local
3 launch_robot.py robot_client=franka_hardware
```

代码块

```
1 # 注意终端1每次关闭后
2 sudo pkill -9 run_server
```

代码块

```
1 # 终端2:起夹爪server (端口 50052) , 可以切换为franka_hand
2
3 conda activate polymetis-local
4 launch_gripper.py gripper=robotiq_2f
```

代码块

```
1 # 终端3:运行后franka会运行到初始位置
2 conda activate polymetis-local
3 cd ~/realRL/gello_software/experiments
4 python launch_nodes.py --robot=fr3 --robot_ip=127.0.0.1
```

代码块

```
1 # 终端4:启动前准备:
2 # - **握紧主臂夹爪**
3 # - 将主臂抬至与 Franka 随动臂 **相近姿态**
4 # 注意: 启动后 Franka 可能快速运动到 leader 位姿, 注意人身安全
5
6 conda activate polymetis-local
7 cd ~/gello_software/experiments
8 python run_env.py --agent=gello
```

注意: 第一次启动时需要进行标定, 在第三个终端进行

1. 第一次使用时, 建议用 `gello_get_offset.py` 做更细的标定:
2. 让遥杆处于与 `--start-joints` 指定关节 (或系统默认 start joints) 尽量接近的位姿 (目视误差不大即可)
3. 运行 offset 工具 (示例命令, 端口需替换为你实际的 `/dev/serial/by-id/...`):

代码块

```
1 conda activate polymetis-local
2 cd ~/gello_software/scripts
3 # 这里的start-joints需要与gello_agent里保持一致, joint-signs控制了关节的运动方向
```

```

4 python3 gello_get_offset.py --start-joints 0 0 0 -1.57 0 1.57 0 --joint-
  signs 1 -1 1 1 1 -1 1 --port /dev/serial/by-id/usb-FTDI_USB__-
  __Serial_Converter_FTAM5DHY-if00-port0
5
6 '''
7 "/dev/serial/by-id/usb-FTDI_USB__-__Serial_Converter_FTAKC4NZ-if00-port0":
  DynamixelRobotConfig(
8     joint_ids=(1, 2, 3, 4, 5, 6, 7),
9     joint_offsets=(
10         0 * np.pi / 2,
11         2 * np.pi / 2,
12         2 * np.pi / 2,
13         2 * np.pi / 2,
14         2 * np.pi / 2,
15         2 * np.pi / 2,
16         5 * np.pi / 4,
17     ),
18     joint_signs=(1, -1, 1, -1, 1, 1, 1),
19     gripper_config=(8, 116, 74),
20 ),
21 '''

```

常见注意事项

1. 尽量不要使用代理（proxy），可能影响 gRPC/ZMQ 通信稳定性。

2. robot server 的 Franka 控制 IP 必须正确：

- 修改：

```
~/fairo/polymetis/polymetis/conf/robot_client/franka_hardware.ya
```

- 设置 `robot_ip: "..."` 为你实际控制 IP（浏览器可访问的控制 IP）

3. gripper server 的 robot_ip 也必须正确：

- 修改： `~/fairo/polymetis/polymetis/conf/gripper/franka_hand.yaml`

- 设置 `robot_ip: "..."` 为你实际控制 IP

2. 发布state状态

代码块

```

1 python scripts/arm_state_zmq_pub.py \
2     --side left \
3     --robot_ip 127.0.0.1 \
4     --gripper_ip 127.0.0.1 \
5     --hz_pub 30

```

3. 发布第三视角相机图像

先获取相机标识：

代码块

```
1 # realsense
2 python scripts/get_serial_num.py
3
4 # lumos
5 ls /dev/v4l/by-id/
```

代码块

```
1 # 修改serial部分
2 python scripts/camera_zmq_pub.py \
3     --name third_d455 --type realsense --serial 941322072906 \
4     --port 7000 --width 640 --height 480 --fps 30 --hz_pub 30 \
5     --out_width 420 --out_height 240 \
6     --jpeg_quality 80
```

4. 发布腕部相机图像

代码块

```
1 # lumos
2 python scripts/camera_zmq_pub.py \
3     --name left_wrist_lumos --type lumos --serial usb-
4     XVisio_Technology_XVisio_vSLAM_250801DR48FB26001173-video-index0 \
5     --port 7001 --width 1280 --height 1280 --fps 60 --hz_pub 30 \
6     --out_width 420 --out_height 240 \
7     --jpeg_quality 80
```

代码块

```
1 # realsense, 同样更换--serial内容
2 python scripts/camera_zmq_pub.py \
3     --name left_wrist_d435 --type realsense --serial 141722078696 \
4     --port 7002 --width 640 --height 480 --fps 30 --hz_pub 30 \
5     --out_width 420 --out_height 240 \
6     --jpeg_quality 80
```

右臂

1. 起gello控制

代码块

```
1 # 终端1:起Polymetis robot server (端口 50051)
2 conda activate polymetis-local
3 launch_robot.py robot_client=franka_hardware
```

代码块

```
1 # 注意终端1每次关闭后
2 sudo pkill -9 run_server
```

代码块

```
1 # 终端2:起夹爪server (端口 50052) , 可以切换为franka_hand
2
3 conda activate polymetis-local
4 launch_gripper.py gripper=robotiq_2f
```

代码块

```
1 # 终端3:运行后franka会运行到初始位置
2 conda activate polymetis-local
3 cd ~/realRL/gello_software/experiments
4 python launch_nodes.py --robot=fr3 --robot_ip=127.0.0.1
```

代码块

```
1 # 终端4:启动前准备:
2 # - **握紧主臂夹爪**
3 # - 将主臂抬至与 Franka 随动臂 **相近姿态**
4 # 注意: 启动后 Franka 可能快速运动到 leader 位姿, 注意人身安全
5
6 conda activate polymetis-local
7 cd ~/gello_software/experiments
8 python run_env.py --agent=gello
```

2. 发布state状态

代码块

```
1 python scripts/arm_state_zmq_pub.py \
```

```
2 --side right \  
3 --robot_ip 127.0.0.1 \  
4 --gripper_ip 127.0.0.1 \  
5 --hz_pub 30
```

3. 发布腕部相机图像

代码块

```
1 python scripts/camera_zmq_pub.py \  
2 --name right_wrist_lumos --type lumos --serial usb-  
XVisio_Technology_XVisio_vSLAM_250801DR48FB26001218-video-index0 \  
3 --port 7003 --width 1280 --height 1280 --fps 60 --hz_pub 30 \  
4 --out_width 420 --out_height 240 \  
5 --jpeg_quality 80
```

代码块

```
1 python scripts/camera_zmq_pub.py \  
2 --name right_wrist_d435 --type realsense --serial 141722078696 \  
3 --port 7004 --width 640 --height 480 --fps 30 --hz_pub 30 \  
4 --out_width 420 --out_height 240 \  
5 --jpeg_quality 80
```

左臂：启动collect_data.sh

配置右臂主机 IP

打开并修改：

- `serl_robot_infra/franka_env/envs/zmq_dual_observer_obs.py` 里 `GelloDualObserverConfig.RIGHT_PC_IP`

确保它等于右臂主机的实际 IP（例如 `192.168.120.42`）。

准备 task 文件

在左臂主机创建：

- `experiments/gello_dual_observer/tasks.json`
- `experiments/gello_dual_observer/subtasks.json`

最小示例：

tasks.json

代码块

```
1  {
2    "dummy_task": "record a short bimanual segment"
3  }
```

subtasks.json

代码块

```
1  {
2    "dummy_task": ["press b to finish and save"]
3  }
```

启动采集

在左臂主机 `/home/ubuntu/realRL` 运行：

代码块

```
1  python scripts/collect_data.py \
2    --exp_name=gello_dual_observer \
3    --task_dir=experiments/gello_dual_observer \
4    --task_name=dummy_task \
5    --output_dir=./demo_data_gello_dual
```

代码块

```
1  # 或者修改collect_data.sh的参数，直接使用
2  .\collect_data.sh
```

采集时的按键语义

`collect_data.py` 只有在 `done=True` 时才会落盘 Parquet。

- 按 **b**：subtask success；当 subtasks 全部完成时 episode 结束并 **保存**。
- 按 **c**：任务失败，episode 结束；默认会保存 failed（脚本默认 `--save_failed=True`）。
- 按 **a**：force reset，episode 结束但 **不保存**（abandon）。

输出目录结构：

- `./demo_data_gello_dual/<task_name>/<task_name>_epXXXX.parquet`

数据处理（Parquet → 轻量导出）

采集得到的 Parquet 可用本仓库脚本做**本地后处理**：可选滤除静止帧、生成状态/动作曲线图，并导出**每路相机单独 MP4** + `*_meta.json` + **去掉图像列的轻量 Parquet**（便于上传或二次处理）。

脚本： `scripts/process_data_local.sh`

常用命令（在左臂主机、项目根目录）：

代码块

```
1  # 处理某个任务的全部episode
2  ./scripts/process_data_local.sh {task_name}
3
4  # 处理特定几个episode
5  ./scripts/process_data_local.sh {task_name} 1,2,3
6
7  # 关闭静止帧过滤
8  FILTER=false ./scripts/process_data_local.sh {task_name}
```

这一步结束后可以查看数据分布的画图结果

训练过程

数据上传：

处理后的数据上传至服务器，记得更换为自己的数据存储路径

代码块

```
1  scp -r xxx.zip  eva14:/share/chenshuaiwen-local/dual_franka_data_raw
```

代码块

```
1  scp -r xxx.zip  123.183.193.192:/mnt/public1/chenshuaiwen/dual_franka_data_raw
```

解压对应zip

代码块 unzip xxx.zip

数据格式转换

eva:

代码块

```
1 uv run python dual_scripts/convert_dual_franka_data_to_lerobot.py --input_dirs=  
  <你的原始目录> --repo_id=/share/chenshuaiwen-local/.cache/hf_home/dual_franka/任  
  务名
```

无问:

代码块

```
1 uv run python dual_scripts/convert_dual_franka_data_to_lerobot.py --input_dirs=  
  <你的原始目录> --repo_id=/mnt/public1/chenshuaiwen/.cache/hf_home/dual_franka/任  
  务名
```

注意：如果想把多个数据集混合训练，建议在这一步进行

代码块

```
1 cd /home/chenshuaiwen/openpi  
2  
3 uv run python dual_scripts/convert_dual_franka_data_to_lerobot.py \  
4   --input_dirs=/path/ds1 \  
5   --input_dirs=/path/ds2 \  
6   --input_dirs=/path/ds3 \  
7   --input_dirs=/path/ds4 \  
8   --input_dirs=/path/ds5 \  
9   --input_dirs=/path/ds6 \  
10  --input_dirs=/path/ds7 \  
11  --input_dirs=/path/ds8 \  
12  --repo_id=/share/chenshuaiwen-  
    local/.cache/hf_home/dual_franka/handover_mix_8sets
```

config修改

1. 对齐输入格式

src/openpi/policies下写个policy做输入输出格式的变换

2. 改写config文件

需要一个data process config， 一个train config

train config写了数据路径和base model路径，还有一些训练参数

注意ckpt不要保存在home下，有内存限额，在config的

checkpoint_base_dir: 修改

norm计算

代码块

```
1 uv run scripts/compute_norm_stats.py --config-name {your_config_name}
```

开训

注意开tmux，现在的tmux和exp_name相同

代码块

```
1 tmux new -s exp_name
2
3 tmux attach -t exp_name
```

注意改exp_name

JAX会默认使用可见的所有卡，所以需要修改可见卡数为当前可用的

代码块

```
1 CUDA_VISIBLE_DEVICES=0 XLA_PYTHON_CLIENT_MEM_FRACTION=0.9 uv run
  scripts/train.py {your_config_name} --exp-name={your_exp_name} --overwrite
```

拷贝模型

1. 将norm放入对应ckpt的assets，注意别放错
2. 删除ckpt中的train_status文件夹
3. scp到本地，建议使用校园网环境进行

模型部署

openpi部分

openpi里起server，openpi的配置与前面相同，

- serve_policy 里面可以指定端口号，默认prompt等

代码块

```
1  # openpi下,注意这里指定config和路径
2  # 注意ckpt的assets下应有一个norm_stats.json
3  uv run scripts/serve_policy_dual.py policy:checkpoint --policy.config=
    {your_config_name} --policy.dir={your_policy_dir}
```

- 目前修改了wesocket_policy_server，对齐eval输出和模型输入的接口

```
uv run scripts/serve_policy_dual.py policy:checkpoint --
policy.config=pi05_dual_franka_finetune_high_lumos --
policy.dir=/home/ubuntu/qiuyi/ckpts/handover_third_lumos_400/
```

相机发布

左臂

发布第三视角相机图像

先获取相机标识：

代码块

```
1  # realsense
2  python scripts/get_serial_num.py
3
4  # lumos
5  ls /dev/v4l/by-id/
```

代码块

```
1  # 修改serial部分
2  python scripts/camera_zmq_pub.py \
3  --name third_d455 --type realsense --serial 941322072906 \
4  --port 7000 --width 640 --height 480 --fps 30 --hz_pub 30 \
5  --out_width 420 --out_height 240 \
6  --jpeg_quality 80
```

发布腕部相机图像

1

代码块

```
1 # lumos
2 python scripts/camera_zmq_pub.py \
3     --name left_wrist_lumos --type lumos --serial usb-
  XVisio_Technology_XVisio_vSLAM_250801DR48FB26001173-video-index0 \
4     --port 7001 --width 1280 --height 1280 --fps 60 --hz_pub 30 \
5     --out_width 420 --out_height 240 \
6     --jpeg_quality 80
```

代码块

```
1 # realsense, 同样更换--serial内容
2 python scripts/camera_zmq_pub.py \
3     --name left_wrist_d435 --type realsense --serial 141722078696 \
4     --port 7002 --width 640 --height 480 --fps 30 --hz_pub 30 \
5     --out_width 420 --out_height 240 \
6     --jpeg_quality 80
```

右臂

发布腕部相机图像

1

代码块

```
1 python scripts/camera_zmq_pub.py \
2     --name right_wrist_lumos --type lumos --serial usb-
  XVisio_Technology_XVisio_vSLAM_250801DR48FB26001218-video-index0 \
3     --port 7003 --width 1280 --height 1280 --fps 60 --hz_pub 30 \
4     --out_width 420 --out_height 240 \
5     --jpeg_quality 80
```

代码块

```
1 python scripts/camera_zmq_pub.py \
2     --name right_wrist_d435 --type realsense --serial 141722078696 \
3     --port 7004 --width 640 --height 480 --fps 30 --hz_pub 30 \
```



```
4 --out_width 420 --out_height 240 \  
5 --jpeg_quality 80
```

eval

本地跑eval，使用realRL `scripts/eval_policy.sh`

代码块

```
1 ./run_server.sh
```

代码块

```
1 # realRL, 默认使用lumos相机  
2 ./scripts/eval_policy.sh {task_name} 20  
3 # --zmq_policy_image_keys=left_wrist_lumos,right_wrist_lumos,third_d455  
4 # 可以使用这个参数进行相机修改, 建议写在sh里面
```

handover_high